

NUCLEO-G491RE

3 Fazlı BLDC Komutasyon Simülasyonu

STM32CubeMX + STM32CubeIDE + ST-LINK VCP + CoolTerm

Proje adı: staj_motor_simule_g491

Kart: NUCLEO-G491RE

MCU: STM32G491RET6

Seri arayüz: LPUART1 / ST-LINK Virtual COM Port

Tarih: 02 Eylül 2026

Bu doküman, projeyi sıfırdan yeniden oluşturmak, kodun mantığını anlamak, karta yazmak ve CoolTerm'de sonuçları yorumlamak için hazırlanmıştır.

İçindekiler

1 Amaç ve güvenlik sınırı	2
2 Gerekli donanım ve yazılım	2
2.1 Gereksinimler	2
2.2 Nucleo kartı neden işimizi kolaylaştırır?	2
3 STM32CubeMX ile sıfırdan proje oluşturma	2
3.1 Kart seçimi	2
3.2 LPUART alanı kilitli veya gri görünürse	3
3.3 SYS: Serial Wire debug	3
3.4 LPUART1: bilgisayara seri veri gönderme	3
3.5 Üç faz GPIO çıkışı	3
3.6 Clock ve kod üretimi	4
4 STM32CubeIDE: kodun çalışma mantığı	4
4.1 Faz durumları	4
4.2 Altı adımlı komutasyon tablosu	4
4.3 Set_Phase: HIGH, LOW ve FLOAT uygulanması	5
4.4 Motor_Advance, SysTick ve UART	5
5 Derleme, karta yazma ve debug	6
6 CoolTerm ile UART verisini görme	6
7 CoolTerm Chart: ayar ve yorum	6
7.1 Neden dört trace oluşur?	6
7.2 Temiz grafik için ayar	7
7.3 Grafiği yorumlama	7
7.4 Hem okunaklı metin hem kusursuz chart istenirse	7
8 Kontrol listesi ve sonraki adımlar	7
8.1 Başarı kontrol listesi	7
8.2 Geliştirme sırası	8

1 Amaç ve güvenlik sınırı

Bu uygulama gerçek bir BLDC motoru döndürmez. Bunun yerine üç GPIO pini üzerinde BLDC'nin altı adımlı (six-step) komutasyon sırasını canlandırır. Her **200 ms** sonunda A, B ve C fazının durumu değiştirilir; aynı bilgi LPUART1 ile bilgisayara gönderilir ve CoolTerm'de metin veya grafik olarak izlenir.

Güvenlik: PA4, PB4 ve PB5 yalnızca 3.3 V mantık çıkışlarıdır. BLDC motoru, motor fazını veya MOSFET kapısını bu pinlere doğrudan bağlama. Gerçek sistemde 3-faz inverter/gate driver, akım ölçümü, koruma ve PWM gerekir. Bu çalışma yalnızca komutasyon mantığını güvenli biçimde öğretir.

Öğrenilen veri zinciri şöyledir:

Komutasyon tablosu -> GPIO A/B/C -> LPUART1 TX
-> ST-LINK Virtual COM Port -> CoolTerm

2 Gerekli donanım ve yazılım

2.1 Gereksinimler

- NUCLEO-G491RE geliştirme kartı
- Veri aktarımını destekleyen micro-USB kablo
- macOS çalıştıran bilgisayar
- STM32CubeMX, STM32CubeIDE, CoolTerm
- İsteğe bağlı: STM32CubeProgrammer

2.2 Nucleo kartı neden işimizi kolaylaştırır?

Kart üzerindeki ST-LINK V3E iki ayrı iş yapar:

1. **SWD:** Firmware yazma ve hata ayıklama,
2. **Virtual COM Port (VCP):** UART verisini USB üzerinden Mac'e taşıma.

Bu yüzden ek USB-UART dönüştürücü ve DFU modu gerekmez.

```
Mac + CoolTerm
| micro-USB veri kablosu
v
NUCLEO üzerindeki ST-LINK V3E
|-- SWD <-> PA13 (SWDIO), PA14 (SWCLK): programlama/debug
|-- VCP <-> PA2 (LPUART1_TX), PA3 (LPUART1_RX): UART
v
STM32G491RET6
```

3 STM32CubeMX ile sıfırdan proje oluşturma

3.1 Kart seçimi

1. STM32CubeMX'i aç.
2. Access to Board Selector seçeneğini seç.
3. Arama alanına NUCLEO-G491RE yaz.
4. Kartı seç ve Start Project düğmesine bas.

Kart seçicisini kullanmak önemlidir: CubeMX, kartın ST-LINK ve Virtual COM Port bağlantılarını tanır. Yalnız MCU seçilirse fiziksel kart bağlantılarını senin doğrulaman gerekir.

3.2 LPUART alanı kilitli veya gri görünürse

Kart şablonu bazı pinleri önceden ayırmış olabilir. Sıfırdan açılmış projede LPUART1 modu seçilemiyorsa Pinout menüsünden Clear Pinout veya ilgili Reset Configuration işlemini kullanıp aşağıdaki ayarları yeniden oluşturun. Bu işlem mevcut projede rastgele yapılmamalıdır; önceki pin ayarlarını siler.

3.3 SYS: Serial Wire debug

System Core > SYS ekranında:

Debug = Serial Wire

Bu seçim SWDIO (PA13) ve SWCLK (PA14) pinlerini ST-LINK programlayıcısına ayırır. Bu pinleri başka GPIO işlevi için kullanma. No Debug seçimi, debug ve tekrar programlamayı gereksiz derecede zorlaştırır.

3.4 LPUART1: bilgisayara seri veri gönderme

Connectivity > LPUART1 ekranında aşağıdaki ayarları yap:

Alan	Değer
Mode	Asynchronous
Baud Rate	115200
Word Length	8 Bits
Parity	None
Stop Bits	1
Hardware Flow Control	Disable / None
Communication Mode	TX/RX

Pinout ekranında şu eşleme görünmelidir:

Pin	Fonksiyon	Anlamı
PA2	LPUART1_TX	STM32 → ST-LINK VCP → Mac
PA3	LPUART1_RX	Mac → ST-LINK VCP → STM32

TX (Transmit), STM32'nin dışarı gönderdiği hattır; bu projede STEP satırları PA2 üzerinden gider. **RX** (Receive), STM32'nin dışarıdan aldığı hattır. Henüz komut almıyoruz fakat PA3'ü ayarlamak, ileride bilgisayardan başlat/durdur/hız komutu göndermek için altyapı hazırlar.

Neden 115200 8-N-1? 115200, terminal uygulamalarında yaygın ve bu proje için yeterince hızlıdır. 8-N-1; 8 veri biti, parity yok, 1 stop biti demektir. CoolTerm tarafında da aynı seçilmelidir. CTS/RTS kullanmadığımız için Flow Control kapalıdır.

3.5 Üç faz GPIO çıkışı

Pinout görünümünde şu üç pini GPIO_Output yap ve etiketle:

Pin	User Label	GPIO ayarı
PA4	PHASE_A	Output Push Pull, Low, No pull, Low speed
PB4	PHASE_B	Output Push Pull, Low, No pull, Low speed
PB5	PHASE_C	Output Push Pull, Low, No pull, Low speed

Bu pinler bu projede boş olan üç bağımsız GPIO'dur. **Output Push Pull**, pin HIGH iken aktif 3.3 V; LOW iken aktif GND sürmesini sağlar. Başlangıç seviyesinin Low olması reset sonrası güvenli durum verir. **Low speed** bu yavaş simülasyon için yeterlidir. PA5 ise karttaki LD2 kullanıcı LED'idir; bu projede faz pini değildir.

3.6 Clock ve kod üretimi

CubeMX'in ürettiği yapı HSI ve PLL ile 170 MHz sistem saatini kullanır:

```
HSI = 16 MHz
PLL giriş = 16 / 4 = 4 MHz
VCO = 4 x 85 = 340 MHz
SYSCLK = 340 / 2 = 170 MHz
```

Bu çalışma için Clock Configuration ekranında elle değişiklik yapmak gerekmez. Project Manager ekranında şu değerleri kullan:

Alan	Değer
Project Name	staj_motor_simule_g491
Toolchain / IDE	STM32CubeIDE
Firmware Package	STM32Cube FW_G4
Keep User Code	Etkin

Keep User Code açık olmalıdır. Bu sayede .ioc değişip tekrar Generate Code yapıldığında USER CODE BEGIN/END bloklarına eklenen uygulama kodu korunur.

4 STM32CubeIDE: kodun çalışma mantığı

Kod Core/Src/main.c içindedir. Eklediğin kodu yalnızca USER CODE bloklarına koy. CubeMX'in oluşturduğu MX_GPIO_Init() ve MX_LPUART1_UART_Init() fonksiyonlarını silme.

4.1 Faz durumları

```
typedef uint8_t PhaseState_t;

enum
{
    PHASE_FLOAT = 0U,
    PHASE_LOW = 1U,
    PHASE_HIGH = 2U
};
```

- PHASE_FLOAT: Pin sürülmez, GPIO input/yüksek empedans durumundadır.
- PHASE_LOW: Pin aktif olarak 0 V, yani lojik 0 sürer.
- PHASE_HIGH: Pin aktif olarak 3.3 V, yani lojik 1 sürer.

FLOAT ile LOW aynı şey değildir. FLOAT, gerçek motor sürücüsünde ilgili faz kolunun kapatılmasına karşılık gelen lojik durumun güvenli simülasyonudur.

4.2 Altı adımlı komutasyon tablosu

Adım	PHASE_A	PHASE_B	PHASE_C
1	HIGH (+1)	LOW (-1)	FLOAT (0)
2	HIGH (+1)	FLOAT (0)	LOW (-1)
3	FLOAT (0)	HIGH (+1)	LOW (-1)
4	LOW (-1)	HIGH (+1)	FLOAT (0)
5	LOW (-1)	FLOAT (0)	HIGH (+1)
6	FLOAT (0)	LOW (-1)	HIGH (+1)

Her adımda bir faz HIGH, bir faz LOW, diğer faz FLOAT olur. Altıncı adımdan sonra tekrar ilk adıma dönlür.

```
#define MOTOR_STEP_COUNT 6U
#define MOTOR_STEP_PERIOD_MS 200U

static const uint8_t commutation_table[MOTOR_STEP_COUNT] =
{
    PACK_PHASES(PHASE_HIGH, PHASE_LOW, PHASE_FLOAT),
    PACK_PHASES(PHASE_HIGH, PHASE_FLOAT, PHASE_LOW),
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH, PHASE_LOW),
    PACK_PHASES(PHASE_LOW, PHASE_HIGH, PHASE_FLOAT),
    PACK_PHASES(PHASE_LOW, PHASE_FLOAT, PHASE_HIGH),
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW, PHASE_HIGH)
};
```

PACK_PHASES, üç faz durumunu ikişer bit halinde tek byte'ta saklar. UNPACK_PHASE ise seçilen adımdan A/B/C bilgisini tekrar ayırır.

4.3 Set_Phase: HIGH, LOW ve FLOAT uygulanması

```
static void Set_Phase(GPIO_TypeDef *port, uint16_t pin,
                    PhaseState_t state)
{
    GPIO_InitTypeDef gpio = {0};
    gpio.Pin = pin;
    gpio.Pull = GPIO_NOPULL;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;

    if (state == PHASE_FLOAT)
    {
        gpio.Mode = GPIO_MODE_INPUT;
        HAL_GPIO_Init(port, &gpio);
    }
    else
    {
        gpio.Mode = GPIO_MODE_OUTPUT_PP;
        HAL_GPIO_Init(port, &gpio);
        HAL_GPIO_WritePin(port, pin,
            (state == PHASE_HIGH) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

- HIGH: pin Output Push Pull olur, sonra GPIO_PIN_SET yazılır.
- LOW: pin Output Push Pull olur, sonra GPIO_PIN_RESET yazılır.
- FLOAT: pin input moda alınır; pin artık aktif sürülmez.

4.4 Motor_Advance, SysTick ve UART

Motor_Advance() güncel adımı seçer, A/B/C pinlerini günceller ve aynı bilgiyi LPUART1 üzerinden CoolTerm'e yollar:

```
STEP 1 | A: 1 | B: -1 | C: 0
STEP 2 | A: 1 | B: 0 | C: -1
STEP 3 | A: 0 | B: 1 | C: -1
STEP 4 | A: -1 | B: 1 | C: 0
STEP 5 | A: -1 | B: 0 | C: 1
STEP 6 | A: 0 | B: -1 | C: 1
```

```
const uint32_t now_ms = HAL_GetTick();
if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS)
{
    last_step_ms += MOTOR_STEP_PERIOD_MS;
    Motor_Advance();
}
```

`HAL_GetTick()` milisaniye sayacıdır. Periyot 200 ms olduğu için tam altı adımlı döngü $6 \times 200 = 1200$ ms, yani 1.2 s sürer.

5 Derleme, karta yazma ve debug

CubeIDE’de proje adına tıkla ve çekiç simgesine bas. Beklenen çıktı:

```
Build Finished. 0 errors, 0 warnings.
```

Derleme sonunda şu dosya oluşur:

```
Debug/staj_motor_simule_g491.elf
```

Kart USB ile bağlıyken CubeIDE içinden Run/Debug ile yükleme yapılabilir. ST-LINK firmware’i SWD üzerinden yazar ve MCU’yu resetler. Kartı DFU moduna alma gerekmez.

Eğer CubeIDE ST-Link Server is required to launch the debug session uyarısını verirse bu uygulama kodu hatası değildir; macOS’ta ST-LINK Server bileşeni eksiktir. ST-LINK Server’i kurup IDE’yi yeniden aç veya STM32CubeProgrammer’da ST-LINK/SWD bağlantısını seçip `.elf` dosyasını programla.

6 CoolTerm ile UART verisini görme

CoolTerm’de Options ekranını aç ve şu değerleri gir:

Alan	Değer
Serial Port	/dev/cu.usbmodem...
Baudrate	115200
Data Bits	8
Parity	None
Stop Bits	1
Flow Control	None

Sondaki port numarası her bağlantıda değişebilir; önemli olan NUCLEO/ST-LINK’e ait `/dev/cu.usbmodem...` girdisidir. Connect düğmesine basınca başlık ve STEP 1–6 verileri yaklaşık her 200 ms’de bir görünmelidir.

Metin okunmuyorsa ilk kontrol baud değeridir: hem CubeMX hem CoolTerm **115200** olmalıdır. Sonra 8-N-1 ve Flow Control=None değerlerini kontrol et. Port görünmüyorsa veri taşıyan USB kablosu kullan ve başka terminalin aynı portu açık tutmadığından emin ol.

7 CoolTerm Chart: ayar ve yorum

7.1 Neden dört trace oluşur?

Okunaklı satırın içinde dört sayı vardır:

```
STEP 1 | A: 1 | B: -1 | C: 0
  ^  ^  ^  ^
  |  |  |  | --- Trace 4: C
  |  |  |  | +----- Trace 3: B
  |  |  |  | +----- Trace 2: A
  |  |  |  | +----- Trace 1: STEP 1numaras
```

CoolTerm satırdaki sayıları ayıklar. Bu nedenle 0–6 arasında yükselen ilk eğri faz gerilimi değil, adım numarasıdır.

7.2 Temiz grafik için ayar

1. View > Chart görünümüne geç.
2. Grafik alanında sağ tıkla, Chart Properties seç.
3. Only plot lines with exactly the specified number of data points seçeneğini aç ve değeri 4 yap.
4. Sağ tıkla, Trace Properties seç.
5. Trace 1 için Visible seçeneğini kapat.
6. Trace 2: Phase A; Trace 3: Phase B; Trace 4: Phase C olarak isimlendir.
7. Clear Data ile eski veriyi sil; 1–2 saniye yeni veri biriktir.

Yakınlaştırma için View > Zoom Chart > Zoom In yolunu kullan. Grafik üzerindeyken **Control + trackpad/mouse scroll** da zoom yapar.

7.3 Grafiği yorumlama

Y değeri	Anlamı
+1	Faz GPIO'su aktif olarak 3.3 V (HIGH) sürüyor.
0	Faz FLOAT; pin input/yüksek empedans, aktif sürme yok.
-1	Faz GPIO'su aktif olarak 0 V/GND (LOW) sürüyor.

Beklenen tekrar eden desen:

```
A: +1, +1, 0, -1, -1, 0, ...
B: -1, 0, +1, +1, 0, -1, ...
C: 0, -1, -1, 0, +1, +1, ...
```

Bu eğriler sinüs değil, basamaklı/trapezoidal komutasyon sinyalleridir. Her adımda bir faz HIGH, bir faz LOW, bir faz FLOAT görülmesi doğru sonuçtur.

7.4 Hem okunaklı metin hem kusursuz chart istenirse

İleride iki satır türü göndermek en temiz yöntemdir:

```
STEP 1 | A: 1 | B: -1 | C: 0 // Terminal 11satr
1,-1,0 // Sadece chart 11satr
```

Bu durumda Chart Properties'te yalnızca **3** sayı içeren satırları çiz filtresi seçilir; grafik sadece A/B/C eğrilerinden oluşur.

8 Kontrol listesi ve sonraki adımlar

8.1 Başarı kontrol listesi

- CubeIDE: 0 errors, 0 warnings
- ST-LINK: Firmware karta yazılabilir.
- CoolTerm: /dev/cu.usbmodem..., 115200 8-N-1, Flow Control=None
- Terminal: STEP 1'den STEP 6'ya kadar sıra sürekli tekrar ediyor.
- Chart: A/B/C yalnızca -1, 0, +1 seviyelerinde.

8.2 Geliştirme sırası

1. PA5/LD2 LED'ini her komutasyon döngüsünde değiştir.
2. PC13/B1 butonuyla başlat/durdur ekle.
3. UART RX ile s, p, +, - komutlarını al.
4. HAL_GetTick() yerine timer interrupt kullan.
5. GPIO yerine timer PWM kanallarını öğren.
6. Hall sensörü/back-EMF geri beslemesi ve gerçek gate driver ekle.

Özet: CubeMX donanımı tanımlar; CubeIDE HAL kodunu derler; ST-LINK firmware'i SWD ile karta yazar; LPUART1 veriyi ST-LINK VCP üzerinden Mac'e taşır; CoolTerm ise veriyi terminal ve grafik olarak görünür yapar.